

Hướng dẫn cơ bản về Angular

Giới thiệu

Angular là một framework phát triển ứng dụng web mạnh mẽ và phổ biến, được phát triển bởi Google. Nó giúp xây dựng các ứng dụng web động, hiệu suất cao và dễ bảo trì. Tài liệu này sẽ cung cấp một cái nhìn tổng quan về các thành phần cơ bản của Angular, hướng dẫn cách tạo một dự án Angular nhỏ và cách thực hiện các cuộc gọi API.

Mục lục

1. Các thành phần cơ bản của Angular
 - Components (Thành phần)
 - Modules (Mô-đun)
 - Services (Dịch vụ)
 - Directives (Chỉ thị)
 - Pipes (Bộ lọc)
2. Tạo một dự án Angular nhỏ
 - Cài đặt Angular CLI
 - Tạo dự án mới
 - Chạy ứng dụng
3. Gọi API trong Angular
 - Sử dụng HttpClientModule
 - Thực hiện GET Request
 - Xử lý dữ liệu trả về
4. Kết luận

Components (Thành phần)

Components là các khối xây dựng cơ bản của ứng dụng Angular. Mỗi component kiểm soát một phần giao diện người dùng (UI) trên màn hình, được gọi là view. Một component bao gồm:

- **Một lớp TypeScript:** Chứa logic của component, dữ liệu và các phương thức để tương tác với view.
- **Một template HTML:** Định nghĩa cấu trúc UI của component.
- **Một stylesheet CSS (tùy chọn):** Định nghĩa kiểu dáng cho component.

Ví dụ về một component cơ bản:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-my-component',
  templateUrl: './my-component.component.html',
  styleUrls: ['./my-component.component.css']
})
export class MyComponent {
  title = 'Chào mừng đến với Angular!';
}
```

```
<!-- my-component.component.html -->
<h1>{{ title }}</h1>
```

Modules (Mô-đun)

Angular Modules (hay NgModule) là các container dùng để nhóm các thành phần liên quan như components, services, directives và pipes. Mỗi ứng dụng Angular có ít nhất một root module, thường được đặt tên là **AppModule**. Modules giúp tổ chức ứng dụng thành các khối chức năng, dễ quản lý và tái sử dụng.

Ví dụ về một AppModule :

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

import { AppComponent } from './app.component';
import { MyComponent } from './my-component/my-component.component';

@NgModule({
  declarations: [
    AppComponent,
    MyComponent
  ],
  imports: [
    BrowserModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

```

    ],
    imports: [
        BrowserModule
    ],
    providers: [],
    bootstrap: [AppComponent]
  })
  export class AppModule { }

```

Services (Dịch vụ)

Services là các lớp TypeScript dùng để cung cấp các chức năng không liên quan trực tiếp đến UI, chẳng hạn như lấy dữ liệu từ server, xác thực người dùng, hoặc lưu trữ dữ liệu cục bộ. Services được thiết kế để có thể tái sử dụng và được cung cấp (injected) vào các components hoặc services khác thông qua cơ chế Dependency Injection của Angular.

Ví dụ về một service đơn giản để lấy dữ liệu:

```

import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class DataService {
  constructor() { }

  getHelloMessage(): string {
    return 'Xin chào từ DataService!';
  }
}

```

Để sử dụng service này trong một component:

```

import { Component, OnInit } from '@angular/core';
import { DataService } from '../data.service';

@Component({
  selector: 'app-data-display',
  template: `{{ message }}`,
})
export class DataDisplayComponent implements OnInit {
  message: string = '';
}

```

```

constructor(private dataService: DataService) { }

ngOnInit(): void {
  this.message = this.dataService.getHelloMessage();
}
}

```

Directives (Chỉ thị)

Directives là các lớp cho phép bạn thêm hành vi vào các phần tử trong DOM. Angular có ba loại directives chính:

- **Components:** Đây là các directives có template.
- **Structural Directives:** Thay đổi cấu trúc DOM bằng cách thêm hoặc xóa các phần tử (ví dụ: `*ngIf`, `*ngFor`).
- **Attribute Directives:** Thay đổi giao diện hoặc hành vi của một phần tử, component hoặc directive khác (ví dụ: `ngStyle`, `ngClass`).

Ví dụ về `*ngIf` và `*ngFor`:

```

<!-- app.component.html -->
<div *ngIf="showContent">
  <p>Nội dung này sẽ hiển thị nếu showContent là true.</p>
</div>

<ul>
  <li *ngFor="let item of items">{{ item }}</li>
</ul>

```

```

// app.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
})
export class AppComponent {
  showContent = true;
  items = ['Item 1', 'Item 2', 'Item 3'];
}

```

Pipes (Bộ lọc)

Pipes là các hàm cho phép bạn biến đổi dữ liệu hiển thị trong template mà không làm thay đổi dữ liệu gốc. Angular cung cấp nhiều pipes dựng sẵn như `DatePipe`, `CurrencyPipe`, `UpperCasePipe`, `LowerCasePipe`, `DecimalPipe`, v.v. Bạn cũng có thể tạo custom pipes.

Ví dụ về sử dụng pipes:

```
<!-- app.component.html -->
<p>Ngày hiện tại: {{ todayDate | date:"fullDate" }}</p>
<p>Giá sản phẩm: {{ productPrice | currency:"USD":"symbol":"1.2-2" }}<
<p>Tên viết hoa: {{ 'angular' | uppercase }}</p>
```

```
// app.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
})
export class AppComponent {
  todayDate = new Date();
  productPrice = 123.456;
}
```

2. Tạo một dự án Angular nhỏ

Để bắt đầu với Angular, bạn cần cài đặt Node.js và Angular CLI (Command Line Interface). Angular CLI là một công cụ dòng lệnh mạnh mẽ giúp bạn tạo, phát triển và duy trì các ứng dụng Angular.

Cài đặt Angular CLI

Đảm bảo bạn đã cài đặt Node.js (phiên bản 12.x trở lên) và npm (Node Package Manager). Sau đó, bạn có thể cài đặt Angular CLI toàn cục bằng lệnh sau:

```
npm install -g @angular/cli
```

Sau khi cài đặt, bạn có thể kiểm tra phiên bản Angular CLI để đảm bảo cài đặt thành công:

```
ng version
```

Tạo dự án mới

Để tạo một dự án Angular mới, sử dụng lệnh `ng new` theo sau là tên dự án của bạn. Ví dụ, để tạo một dự án có tên `my-angular-app` :

```
ng new my-angular-app
```

CLI sẽ hỏi bạn một số câu hỏi cấu hình, chẳng hạn như có muốn thêm Angular routing hay không và chọn định dạng stylesheet (CSS, SCSS, Sass, Less). Bạn có thể chọn các tùy chọn mặc định hoặc tùy chỉnh theo ý muốn.

Quá trình này sẽ tạo một thư mục mới với tên dự án của bạn và cài đặt tất cả các gói phụ thuộc cần thiết.

Chạy ứng dụng

Sau khi dự án được tạo, điều hướng vào thư mục dự án:

```
cd my-angular-app
```

Sau đó, chạy ứng dụng bằng lệnh `ng serve` :

```
ng serve --open
```

Lệnh `--open` (hoặc `-o`) sẽ tự động mở trình duyệt của bạn tại địa chỉ `http://localhost:4200/` , nơi ứng dụng Angular của bạn đang chạy. Bất kỳ thay đổi nào bạn thực hiện trong mã nguồn sẽ tự động được biên dịch lại và làm mới trình duyệt.

3. Gọi API trong Angular

Trong các ứng dụng web hiện đại, việc tương tác với các API backend để lấy hoặc gửi dữ liệu là rất phổ biến. Angular cung cấp `HttpClientModule` trong gói `@angular/common/http` để thực hiện các yêu cầu HTTP.

Sử dụng HttpClientModule

Đầu tiên, bạn cần `import HttpClientModule` vào `AppModule` của mình. Mở file `src/app/app.module.ts` và thêm dòng sau:

```
import { HttpClientModule } from '@angular/common/http';

@NgModule({
  declarations: [
    // ...
  ],
  imports: [
    BrowserModule,
    HttpClientModule // Thêm HttpClientModule vào đây
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Thực hiện GET Request

Sau khi `HttpClientModule` được import, bạn có thể `inject HttpClient service` vào bất kỳ component hoặc service nào để thực hiện các yêu cầu HTTP. Ví dụ, chúng ta sẽ tạo một service để lấy danh sách người dùng từ một API giả lập (ví dụ: JSONPlaceholder).

1. Tạo một Service để gọi API:

Sử dụng Angular CLI để tạo một service mới:

```
ng generate service user
```

Hoặc viết thủ công file `src/app/user.service.ts` :

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

interface User {
  id: number;
  name: string;
  email: string;
}

@Injectable({
  providedIn: 'root'
})
export class UserService {
  private apiUrl = 'https://jsonplaceholder.typicode.com/users';

  constructor(private http: HttpClient) { }

  getUsers(): Observable<User[]> {
    return this.http.get<User[]>(this.apiUrl);
  }
}

```

Trong đoạn mã trên:

- HttpClient được inject vào constructor.
- apiUrl là URL của API mà chúng ta muốn gọi.
- Phương thức getUsers() sử dụng this.http.get<User[]>(this.apiUrl) để thực hiện một yêu cầu GET và trả về một Observable chứa mảng User .

2. Sử dụng Service trong Component:

Bây giờ, chúng ta sẽ sử dụng UserService trong một component để hiển thị danh sách người dùng. Mở file src/app/app.component.ts (hoặc tạo một component mới):

```

import { Component, OnInit } from '@angular/core';
import { UserService } from '../user.service';

interface User {
  id: number;
  name: string;
}

```



```

    email: string;
  }

@Component({
  selector: 'app-root',
  template: `
    <h1>Danh sách người dùng</h1>
    <ul *ngIf="users.length > 0">
      <li *ngFor="let user of users">
        {{ user.name }} ({{ user.email }})
      </li>
    </ul>
    <p *ngIf="users.length === 0">Đang tải người dùng...</p>
    <p *ngIf="error">Lỗi: {{ error }}</p>
  `,
  styleUrls: ['./app.component.css']
})
export class AppComponent implements OnInit {
  users: User[] = [];
  error: string | null = null;

  constructor(private userService: UserService) { }

  ngOnInit(): void {
    this.userService.getUsers().subscribe({
      next: (data) => {
        this.users = data;
      },
      error: (err) => {
        this.error = err.message;
        console.error('There was an error!', err);
      }
    });
  }
}

```

Trong đoạn mã trên:

- UserService được inject vào constructor của AppComponent .
- Trong ngOnInit , chúng ta gọi phương thức getUsers() của UserService .
- Phương thức subscribe() được sử dụng để đăng ký nhận dữ liệu từ Observable . next xử lý dữ liệu thành công, và error xử lý các lỗi xảy ra trong quá trình gọi API.

Xử lý dữ liệu trả về

Khi `Observable` phát ra dữ liệu (thành công) hoặc lỗi, các hàm `next` và `error` trong `subscribe` sẽ được gọi. Bạn có thể cập nhật trạng thái của component (ví dụ: gán dữ liệu vào biến `users`, hiển thị thông báo lỗi) dựa trên kết quả này.

4. Kết luận

Angular là một framework mạnh mẽ và linh hoạt để xây dựng các ứng dụng web phức tạp. Bằng cách hiểu rõ các thành phần cơ bản như Components, Modules, Services, Directives, Pipes và biết cách tạo dự án, gọi API, bạn đã có một nền tảng vững chắc để bắt đầu phát triển với Angular. Hãy tiếp tục khám phá các tính năng nâng cao khác của Angular để xây dựng những ứng dụng ấn tượng.

References

1. [Anatomy of components - Angular](#)
2. [Introduction to components and templates - Angular](#)
3. [Composition with components - Angular](#)
4. [Angular Components Overview - GeeksforGeeks](#)
5. [Introduction to Angular Components and How to Implement Them?](#)
6. [Angular 13 Components. 1. Understanding Components - Medium](#)
7. [Create a new project - Angular](#)
8. [ng new - Angular](#)
9. [How to Create an Angular Project from Scratch - Medium](#)
10. [How To Create A New Angular Project - YouTube](#)
11. [Installation - Angular](#)
12. [HTTP Client • Overview - Angular](#)
13. [HTTP: Request data from a server - Angular](#)
14. [HttpClient - Angular](#)
15. [Making HTTP requests - Angular](#)

Ví dụ thực tế: Ứng dụng hiển thị danh sách người dùng

Để minh họa các khái niệm đã học, chúng ta đã tạo một ứng dụng Angular đơn giản hiển thị danh sách người dùng từ API JSONPlaceholder. Ứng dụng này bao gồm:

Cấu trúc dự án

```
my-angular-demo/  
├── src/  
│   ├── app/  
│   │   ├── user-list/  
│   │   │   ├── user-list.component.ts  
│   │   │   ├── user-list.component.html  
│   │   │   └── user-list.component.css  
│   │   ├── user.service.ts  
│   │   ├── app.component.ts  
│   │   ├── app.component.html  
│   │   ├── app.component.css  
│   │   └── app.config.ts  
│   ├── main.ts  
└── package.json
```

Các tính năng chính

1. **UserService**: Service để gọi API và lấy dữ liệu người dùng
2. **UserListComponent**: Component hiển thị danh sách người dùng với giao diện đẹp
3. **Xử lý trạng thái**: Loading, error handling và hiển thị dữ liệu
4. **Responsive design**: Giao diện thích ứng với nhiều kích thước màn hình

Kết quả

Ứng dụng hiển thị danh sách 10 người dùng từ API JSONPlaceholder, mỗi người dùng được hiển thị trong một card với thông tin:

- Tên
- Email

- Số điện thoại
- Website

Ứng dụng có giao diện đẹp với header gradient và layout grid responsive.

Các bước tiếp theo

Sau khi đã nắm vững các khái niệm cơ bản của Angular, bạn có thể tiếp tục học các chủ đề nâng cao sau:

1. Routing và Navigation

- Tạo nhiều trang trong ứng dụng
- Điều hướng giữa các trang
- Route parameters và query parameters
- Route guards để bảo vệ các trang

2. Forms và Validation

- Template-driven forms
- Reactive forms
- Form validation
- Custom validators

3. State Management

- Quản lý state với services
- RxJS và Observables nâng cao
- NgRx cho ứng dụng lớn

4. Testing

- Unit testing với Jasmine và Karma
- End-to-end testing với Protractor hoặc Cypress
- Testing components, services và pipes

5. Performance Optimization

- Lazy loading modules
- OnPush change detection strategy
- TrackBy functions
- Bundle optimization

6. Deployment

- Build production
- Deploy lên các platform như Netlify, Vercel, Firebase
- CI/CD với GitHub Actions